

Pandas - II

Employee DataFrame

```
import pandas as pd

df = pd.DataFrame({
    "employee": ["Amit", "Neha", "Raj", "Priya", "Amit", "Neha",
                 "Raj", "Priya", "Karan", "Karan"],
    "department": ["IT", "IT", "HR", "HR", "IT", "IT",
                  "HR", "HR", "Sales", "Sales"],
    "salary": [70000, 85000, 60000, 75000, 90000, 80000,
              65000, 70000, 55000, 65000],
    "region": ["North", "North", "South", "South", "North", "North",
              "South", "South", "West", "West"],
    "performance": [82, 91, 76, 88, 95, 86, 80, 90, 72, 85]
})

print(df)

First, try df.groupby()
```

employee	department	salary	region	performance
Amit	IT	70000	North	82
Neha	IT	85000	North	91
Raj	HR	60000	South	76
Priya	HR	75000	South	88
Amit	IT	90000	North	95
Neha	IT	80000	North	86
Raj	HR	65000	South	80
Priya	HR	70000	South	90
Karan	Sales	55000	West	72
Karan	Sales	65000	West	85

Groupby Commands

- `df.groupby("department")["salary"].count()`
- `df.groupby("region")["salary"].count()`
- `df.groupby("region")["performance"].count()`

Groupby -> **apply()**

- Apply function *func* group-wise and combine the results together.

Groupby -> Transform

- Call function producing a same-indexed DataFrame on each group.
- Returns a DataFrame having the same indexes as the original object filled with the transformed values.
- Transform() is very useful when you want to **compare each employee's salary with their department's average salary.**

Groupby -> Transform

- `df["dept_avg_salary"] = (df.groupby("department")["salary"].transform("mean"))`

employee	department	salary	dept_avg_salary
Amit	IT	50,000	60,000
Ravi	IT	70,000	60,000
Neha	HR	40,000	50,000
Priya	HR	60,000	50,000
Raj	Sales	80,000	80,000

Because `transform()` gives the calculated value **back to every original row belonging to that group**.

Pandas nlargest() Method

- `DataFrame.nlargest(n, columns, keep='first')[source]`
 - Return the first n rows with the largest values in columns, in descending order.
 - The columns that are not specified are returned as well, but not used for ordering.
 - Pandas keeps the **first occurrence** (keep='first') when there is a tie.
- This method is equivalent to `df.sort_values(columns, ascending=False).head(n)`, but **more performant**.
- `nlargest()` is specifically designed to find the largest values efficiently, so it can avoid a complete sort. Therefore: **`nlargest()` is generally faster when you only need the top N values.**

employee	department	salary	region	performance
Amit	IT	70000	North	82
Neha	IT	85000	North	91
Raj	HR	60000	South	76
Priya	HR	75000	South	88
Amit	IT	90000	North	95
Neha	IT	80000	North	86
Raj	HR	65000	South	80
Priya	HR	70000	South	90
Karan	Sales	55000	West	72
Karan	Sales	65000	West	85

- Find the second-highest salary in each department.

```
df.groupby("department")["salary"].apply(lambda x:
x.nlargest(2).min())
```

Lambda is a short way of writing a function. Here, x represents the data that Pandas gives to the function.

```
df.groupby("department")["salary"].apply(lambda  
x: x.nlargest(2).min())
```

IT group



x = [50000, 90000, 70000]



x.nlargest(2)



[90000, 70000]



.min()



70000

Apply() vs Vectorization

- Suppose every employee receives a 10% bonus.

Using apply()

```
df["bonus"] =  
df["salary"].apply(lambda x: x * 0.10)
```

But it processes values by applying a Python function to each element.

Vectorized Approach

```
df["bonus"] = df["salary"] * 0.10
```

This is preferred.

Pandas/NumPy can perform the operation efficiently over the entire column instead of repeatedly invoking a Python-level function.

apply() is useful when the operation **cannot easily be expressed through vectorized Pandas operations**.
a lambda function is a short, one-liner anonymous function used alongside methods like `.apply()` or `.map()` to manipulate data without defining a traditional function using `def`

Explode()

- Explode() method transforms each element of a list-like structure into its own row while **replicating the index and non-exploded column values** across those new rows.
- It is primarily used to unnest, flatten, or normalize datasets where a single cell contains multiple values packaged in a list, tuple, set, or array.

```
df = pd.DataFrame({"student": ["A", "B"], "subjects": [["Math", "Sci"], ["Eng"]]})  
df.explode("subjects")
```

	student	subjects
0	A	Math
0	A	Sci
1	B	Eng

Merge()

- combines two DataFrames or named Series using **SQL-style database joins** based on common columns or indexes.
- By default, it performs an **inner join**, keeping only the rows with matching keys in both datasets
 - Try other join types
- *If you merge two DataFrames on a column and don't specify how=, what happens to rows that exist in only one DataFrame?"*
 - **They're dropped.** Default is how="inner" — a very common silent-data-loss bug in real pipelines.

Merge() (contd..)

```
import pandas as pd
```

```
df1 = pd.DataFrame({'user_id': [1, 2, 3], 'name': ['Alice', 'Bob',  
'Charlie']})
```

```
df2 = pd.DataFrame({'user_id': [2, 3, 4], 'spend': [50, 120, 10]})
```

```
pd.merge(df1, df2, on='user_id', how='inner')
```

	user_id	name	spend
0	2	Bob	50
1	3	Charlie	120

Finding consecutive streak patterns with shift() and cumsum()

in time-series analysis, attendance streaks, login sessions, stock movements, etc.

```
df = pd.DataFrame({  
    "day": [1, 2, 3, 4, 5, 6, 7],  
    "status": [  
        "Up", "Up", "Up",  
        "Down", "Down",  
        "Up", "Up"  
    ]  
})
```

```
df["previous"] = df["status"].shift()
```

shift() moves the values one row down.

day	status	previous
1	Up	NaN
2	Up	Up
3	Up	Up
4	Down	Up
5	Down	Down
6	Up	Down
7	Up	Up

Finding consecutive streak patterns with shift() and cumsum()

in time-series analysis, attendance streaks, login sessions, stock movements, etc.

```
df["status"] != df["status"].shift()
```

```
True  
False  
False  
True  
False  
True  
False
```

These Trues identify **where a new streak starts**.

```
df["streak_id"] = (  
    df["status"] != df["status"].shift()).cumsum()
```

```
True = 1  
False = 0
```

Finding consecutive streak patterns with shift() and cumsum()

Now Pandas treats:

True = 1

False = 0

and calculates cumulative sum:

True → 1

False → 1

False → 1

True → 2

False → 2

True → 3

False → 3

day	status	previous	streak_id
1	Up	NaN	1
2	Up	Up	1
3	Up	Up	1
4	Down	Up	2
5	Down	Down	2
6	Up	Down	3
7	Up	Up	3

Day:	1	2	3	4	5	6	7
Status:	Up	Up	Up	Down	Down	Up	Up
		└───┘		└───┘		└───┘	
Streak:	1			2			3

Whenever the status changes, `streak_id` increases by 1. If the status stays the same, the `streak_id` remains the same.

So:

- **Days 1–3: Up → Streak 1**
- **Days 4–5: Down → Streak 2**
- **Days 6–7: Up → Streak 3**

Finding consecutive streak patterns with `shift()` and `cumsum()`

- This technique is commonly called **streak/group identification** and is useful for questions like:
 - Find consecutive days a student was present.
 - Find consecutive wins/losses.
 - Identify runs of the same machine status.
 - Find periods of continuous "Active" / "Inactive" status.

Idxmax() vs max()

	employee	department	salary	region	performance
0	Amit	IT	70000	North	82
1	Neha	IT	85000	North	91
2	Raj	HR	60000	South	76
3	Priya	HR	75000	South	88
4	Amit	IT	90000	North	95
5	Neha	IT	80000	North	86
6	Raj	HR	65000	South	80
7	Priya	HR	70000	South	90
8	Karan	Sales	55000	West	72
9	Karan	Sales	65000	West	85

- `df['performance'].max()`. -> 95
- Which employee has the highest performance?
- `df.loc[df["performance"].idxmax()]`

`max()` → gives the VALUE
`idxmax()` → gives the INDEX
`loc[]` → gives the ROW

	4
employee	Amit
department	IT
salary	90000
region	North
performance	95

dtype: object